Azure Data Factory

Dynamic Table Extraction Using Lookup, ForEach & Copy Data

Concise study notes • Sections 1–7 • SQL Database to Blob Storage

1. What did we build?

Objective: Retrieve all tables from an Azure SQL Database and copy each table into Azure Blob Storage as a separate CSV file.

Technical concept: Metadata-driven dynamic pipeline. Instead of creating one Copy Data activity per table, the pipeline discovers tables and processes them dynamically.

Pipeline flow: Azure SQL Database (AdventureWorks) → Lookup_All_Tables → ForEach_All_Tables → Copy_Each_Table_To_Blob → Azure Blob Storage (CSV files).

2. Create Azure SQL Database

Create these resources in Azure Portal:

Resource	Example name
Resource Group	rg-retail-dev
SQL logical server	sqlsrv-retail-dev
SQL Database	sqldb-adventureworks-dev

Use the AdventureWorks sample database, which contains tables such as Customer, Product, SalesOrderHeader and SalesOrderDetail.

Terminology: The SQL logical server hosts and manages the database. The database contains the actual tables and records.

3. Create SQL Linked Service

Path: ADF Studio → Manage → Linked services → New → Azure SQL Database
Name: LS_AzureSql_AdventureWorks

Configure the server endpoint, database name and authentication details. Test the connection and save.

Purpose: A Linked Service defines how ADF connects to an external data source or destination.

4. Create SQL Dataset

Path: Author → Datasets → New Dataset → Azure SQL Database
Name: DS_SQL_AdventureWorks_Source
Linked Service: LS_AzureSql_AdventureWorks

Purpose: A Dataset identifies the data ADF accesses through a Linked Service.

Create these dataset parameters for dynamic table extraction:

Parameter	Type
pSchemaName	String
pTableName	String

Set the dataset schema and table properties to:

```
@dataset().pSchemaName
@dataset().pTableName
```

One reusable dataset can now point to different tables by receiving different parameter values.

5. Create Main Pipeline

Path: Author → Pipelines → New Pipeline
Name: PL_SQL_To_Blob_All_Tables

Pipeline structure:

```
PL_SQL_To_Blob_All_Tables
■■■ Lookup_All_Tables
```

```
■■■ ForEach_All_Tables
■■■ Copy_Each_Table_To_Blob
```

The pipeline orchestrates the complete extraction process.

6. Configure Lookup Activity

Activity: Lookup_All_Tables
Purpose: Retrieve table names and schemas from the SQL database.

Property	Value
Source dataset	DS_SQL_AdventureWorks_Source
Use query	Query
First row only	Unchecked

SQL query:

```
SELECT
TABLE_SCHEMA,
TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE';
```

Query meaning: INFORMATION_SCHEMA.TABLES provides table metadata; TABLE_SCHEMA returns the schema; TABLE_NAME returns the table name; BASE TABLE excludes views.

Example output:

TABLE_SCHEMA	TABLE_NAME
SalesLT	Customer
SalesLT	Product
SalesLT	SalesOrderHeader
SalesLT	SalesOrderDetail

With First row only unchecked, Lookup returns multiple records through:

```
@activity('Lookup_All_Tables').output.value
```

Remember: First row only = True returns output.firstRow; False returns output.value (an array of records).

7. Configure ForEach Activity

Activity: ForEach_All_Tables
Purpose: Iterate through every record returned by Lookup.

Under Settings → Items, enter:

```
@activity('Lookup_All_Tables').output.value
```

What is @item()? It represents the current record in the ForEach iteration.

Example current item:

```
{ "TABLE_SCHEMA": "SalesLT", "TABLE_NAME": "Customer" }
```

Pass the current record into the SQL dataset parameters:

Dataset parameter	Dynamic value
pSchemaName	@item().TABLE_SCHEMA
pTableName	@item().TABLE_NAME

ForEach execution modes

Mode	Behaviour
Sequential	Processes one table at a time.
Parallel	Processes multiple tables simultaneously, subject to batch count and available capacity.

For initial learning, sequential execution is easier to understand and debug.

Quick revision

Activity	Responsibility
Lookup	Finds what needs to be processed.
ForEach	Iterates through the returned records.
Copy Data	Performs the actual data movement.

Key takeaway: Lookup + ForEach + parameterized datasets make it possible to build one reusable pipeline that processes multiple SQL tables dynamically.